
JDSSArrow

A SOLID, pluggable reference implementation of JDSS + web monitoring

JDSSArrow - a SOLID, pluggable reference implementation of the Joint Dismounted Soldier System (NATO STANAG 4677, specified in AEP-76, 5 volumes), with a web configuration and monitoring application.

Reference / educational software implementing a published NATO interoperability standard. Not accredited or production tactical software.

1. What JDSSArrow is

JDSSArrow is a SOLID, scalable, pluggable reference implementation of the Joint Dismounted Soldier System (STANAG 4677 / AEP-76), together with a web configuration and monitoring application. It demonstrates the standard end to end: national nodes joining a coalition network, exchanging the seven JDSS message types, rejecting non-compliant traffic, and being observed and managed from a browser dashboard.

It is reference / educational software implementing a published NATO interoperability standard - not accredited or production tactical software.

2. Architecture

Every AEP-76 volume depends only on abstractions (Python Protocols declared in `interfaces.py`). The gateway package is the composition root that wires concrete implementations together via dependency injection. Every seam is a runtime plugin, discovered through `importlib.metadata` entry points, so any piece can be swapped - even for a third-party implementation - without touching the core.

Extension points (each a pluggable Protocol)

- Transport - how frames move between nodes (loopback, UDP multicast).
- Codec - how a JDSSDM message is encoded (xml, json, arrow).
- SecurityProvider - how a payload is authenticated (null, psk / HMAC-SHA256).
- RadioBearer - the loaned-radio abstraction (simulated).
- AddressAllocator - how the network address/multicast group is derived (default).
- ConnectionPolicy - the admit/deny matrix row (matrix, allow_all).
- MessageHandler, MetricsSink, ConfigStore - dispatch, telemetry and persistence seams.

The five volumes map to modules

Vol	AEP-76 subject	Python module
I	Security	<code>jdssarrow.security</code>
II	Data Model (JDSSDM)	<code>jdssarrow.datamodel</code>
III	Loaned Radio	<code>jdssarrow.loanedradio</code>
IV	Information Exchange Mechanism	<code>jdssarrow.iem</code>
V	Network Access	<code>jdssarrow.networkaccess</code>

A node is a `JdssGateway` (the composed stack) driven by a `SoldierNode` (the high-level client API: `start`, `identify`, `presence`, `report_contact`, `request_casevac`, `chat`, `stop`).

3. The web application

A FastAPI backend owns one gateway for its lifetime and exposes REST config endpoints, a monitoring snapshot, a Prometheus /metrics endpoint, a WebSocket that streams live message events, and an Apache Arrow IPC dump of recent telemetry. A React + TypeScript single-page app renders the dashboard. The backend also serves the built dashboard, so one process delivers both the API and the UI on port 8000.

Dashboard tabs

- Dashboard - KPI strip (status, uptime, peers, messages, rejected, coalition version, classification), health, connected peers, the AEP-76 volumes, message injection and a live feed.
- Configuration - the hot-reloadable pluggable config editor, node identity/network, and the capability matrix (per-message-type receive/emit permissions).
- Connections & Policy - local connection management, the gossip-distributed coalition policy (with Ed25519-signed status), and the live connection matrix.
- Simulation - start/stop a live roster of JDSS-compliant clients on this node's network.
- Connect a Client - live join coordinates plus copy-paste onboarding for every client type.
- Logs - the application log and the per-message audit log, live.
- Documentation - links to these PDF guides.

4. Interoperability simulator

jdssarrow.simulator spawns a roster of role-based clients, each driving a real gateway, so every message is conformant by construction. Client types are themselves plugins (the jdssarrow.profiles entry-point group).

Client	Role	Characteristic messages
rifleman	dismounted soldier	Presence, Chat
teamleader	team leader	Identification, Presence, Chat, Contact, CASEVAC
medic	combat medic	Presence, Chat; answers CASEVAC and repositions
scout	recce	Contact, Sketch, Presence
observer	forward observer / JTAC	Overlay (control measures), Contact
sensor	UAV / automated sensor	Contact (detections), Presence
commandpost	HQ	Identification, Chat (orders); builds the COP
atak	ATAK end-user device	Presence, Chat, Contact, Overlay
vehicle	mounted C2 platform	Identification, Presence, Contact

Rejecting non-compliant traffic

A rogue client exercises each rejection boundary; the network drops its traffic while legitimate exchange is unaffected. Each rogue maps to a volume's defence:

- wrong_key (Vol I) - unauthorised PSK, so the HMAC verify fails.
- garbage (Vol IV) - not a JDSS frame, so framing rejects it.
- insider (Vol II) - leaked key but a bad payload, so the codec/schema rejects it.

Each node independently records who it accepted traffic from - one row of the N x N connection matrix. A rejected rogue's column is all-zero (nobody accepts it), even though its own row may be populated (it can still receive).

Managing connections and coalition policy

Beyond observing, the matrix manages connections. Each node carries a connection policy (its row of the coalition admit/deny matrix), enforced on ingest after security and schema but before dispatch. For a network-wide rule, one node is designated the policy authority: it owns the coalition policy and distributes it over an HMAC-authenticated, versioned gossip channel; every node applies it under its own local policy (a peer must be allowed by both). Coalition updates are additionally signed with the authority's Ed25519 private key, so a coalition-key holder without the private key cannot forge or replay an update.

5. Logging and observability

- Application log - standard Python logging on the jdssarrow logger (lifecycle, warnings, errors), captured into a bounded ring (GET /api/logs/app).
- Message audit log - one entry per message with direction (in/out), disposition (accepted/rejected) and, for rejections, the reason: framing, security, codec, policy, capability or duplicate (GET /api/logs/messages).
- Prometheus /metrics - counters that use the same reason tokens as the audit log, so metrics and the audit trail always agree.

6. Technology stack

Layer	Technology
Language / runtime	Python 3.14 (asyncio)
Web framework	FastAPI + Uvicorn (ASGI)
Data / validation	Pydantic v2, pydantic-settings
Serialisation	lxml (XML), Apache Arrow (pyarrow) for telemetry
Security	cryptography (Ed25519), HMAC-SHA256
Metrics	prometheus-client
Frontend	React 18 + TypeScript, built with Vite
Packaging	uv + hatchling; container via Docker

7. Quick start

```
uv venv && uv pip install -e "[dev]"

# run the test suite (incl. two-node loopback e2e round-trip)
pytest -q

# demo: two nodes exchange Presence + CASEVAC over real UDP multicast
jdssarrow run --config examples/node-a.yaml # terminal 1
jdssarrow run --config examples/node-b.yaml # terminal 2

# interoperability simulator (prints a compliance report)
jdssarrow simulate

# web config + monitoring backend -> http://localhost:8000
uvicorn jdssarrow.web.app:app --reload

# or run the whole thing in Docker
./deploy.sh
```